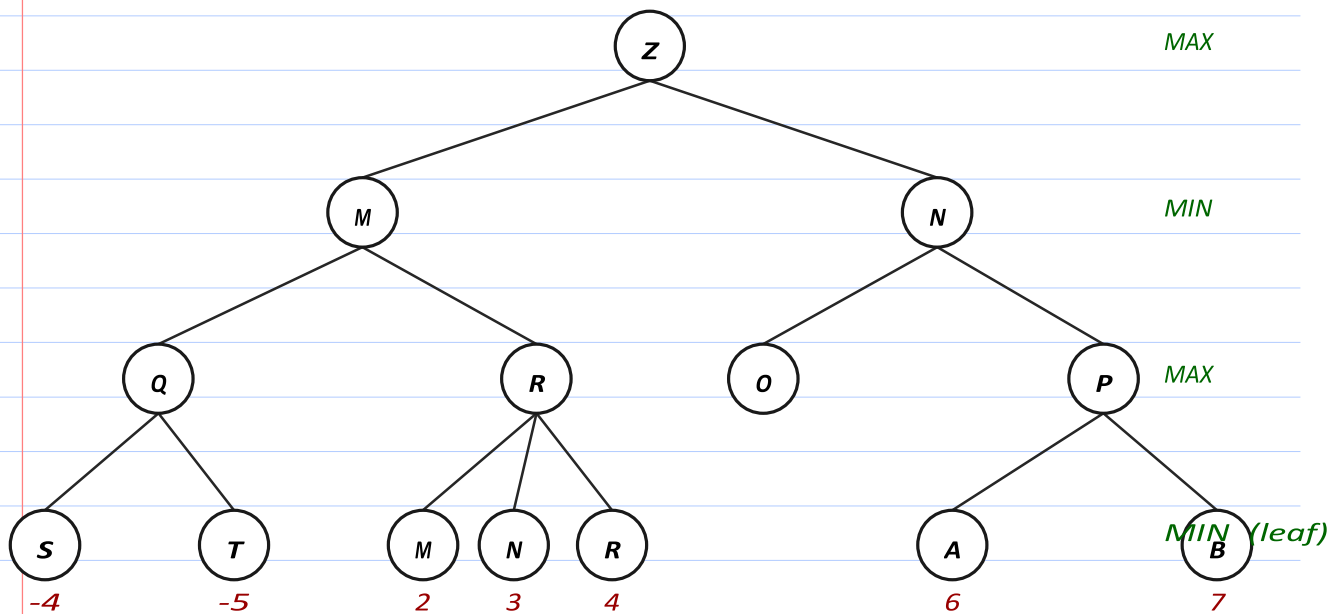


1

Q1. Working of Min-Max Algorithm**Step-by-step Evaluation:**

Step 1: Identify leaf nodes with values:

$$S=-4, T=-5, M=2, N=3, R=4, A=6, B=7$$

Step 2: Level 3 is MAX. Each MAX node picks max of children: $Q = \max(-4, -5) = -4$
 $R = \max(2, 3, 4) = 4$ $O = 5$ (single child)

$$P = \max(6, 7) = 7$$

Step 3: Level 2 is MIN. Each MIN node picks min of children: $M = \min(Q, R) = \min(-4, 4) = -4$

$$N = \min(O, P) = \min(5, 7) = 5$$

Step 4: Level 1 (root Z) is MAX:

$$Z = \max(M, N) = \max(-4, 5) = 5$$

==> Optimal value at root Z

Question no 2:

[1]

Os



```
def minimax(node, depth, is_max):
    if depth == 0 or node.is_terminal():
        return node.value

    if is_max:
        max_val = -float('inf')
        for child in node.children:
            val = minimax(child, depth-1, False)
            max_val = max(max_val, val)
        return max_val
    else:
        min_val = float('inf')
        for child in node.children:
            val = minimax(child, depth-1, True)
            min_val = min(min_val, val)
        return min_val
```

Task # 1 - Q3. 20 Important Questions from Min-Max Code

Q1. What does the minimax algorithm do?

Ans: It finds the optimal move for a player assuming the opponent also plays optim

Q2. What are the two roles in minimax?

Ans: Maximizer (tries to get highest score) and Minimizer (tries to get lowest sco

Q3. What is the base case in minimax recursion?

Ans: When depth==0 or the node is a terminal (leaf) node; return its heuristic val

Q4. Write the syntax of the minimax function:

Ans: def minimax(node, depth, isMaximizing): if depth==0 or is_terminal(node): return

evaluate(node) if isMaximizing:

best = -infinity for child in get_children(node):

best = max(best, minimax(child, depth-1, False)) return best else:

best = +infinity for child in get_children(node):

best = min(best, minimax(child, depth-1, True)) return best

Q5. What does isMaximizing=True mean?

Ans: It is the Maximizer's turn; the function returns the maximum value among chil

Q6.

What is the time complexity of minimax?

Ans: $O(b^d)$ where b =branching factor and d =depth of the tree.

Q7. What is the space complexity of minimax?

Ans: $O(b*d)$ — proportional to the depth of the tree due to recursion stack.

Q8. What does evaluate(node) return?

Ans: A heuristic/utility score representing how good the position is for the Maxim

Q9. How is minimax used in chess?

Ans: It searches possible moves to a certain depth and picks the move with the bes

Q10. Why does Minimizer choose minimum?

Ans: It represents the opponent who wants to minimize the Maximizer's score.

Q11. What is a terminal node?

Ans: A node where the game ends (win/loss/draw) or the search depth limit is reached.

Q12. What is the role of depth parameter?

Ans: It limits how deep the recursion goes to avoid infinite loops / manage perfor

Q13. What happens when depth reaches 0?

Ans: The function stops recursing and returns the static evaluation of that node.

Q14. What is -infinity used for?

Ans: Initial value for the Maximizer so any real child value will be larger and ac

Q15. What is +infinity used for?

Ans: Initial value for the Minimizer so any real child value will be smaller and a

Q16. Does minimax guarantee a win?

Ans: It guarantees the best possible outcome given both players play optimally.

Q17. Is minimax suitable for games with chance?

Ans: No, for games with chance (dice) we use ExpectiMinimax instead.

Q18. What is get_children(node)?

Ans: A function returning all possible game states reachable in one move from the

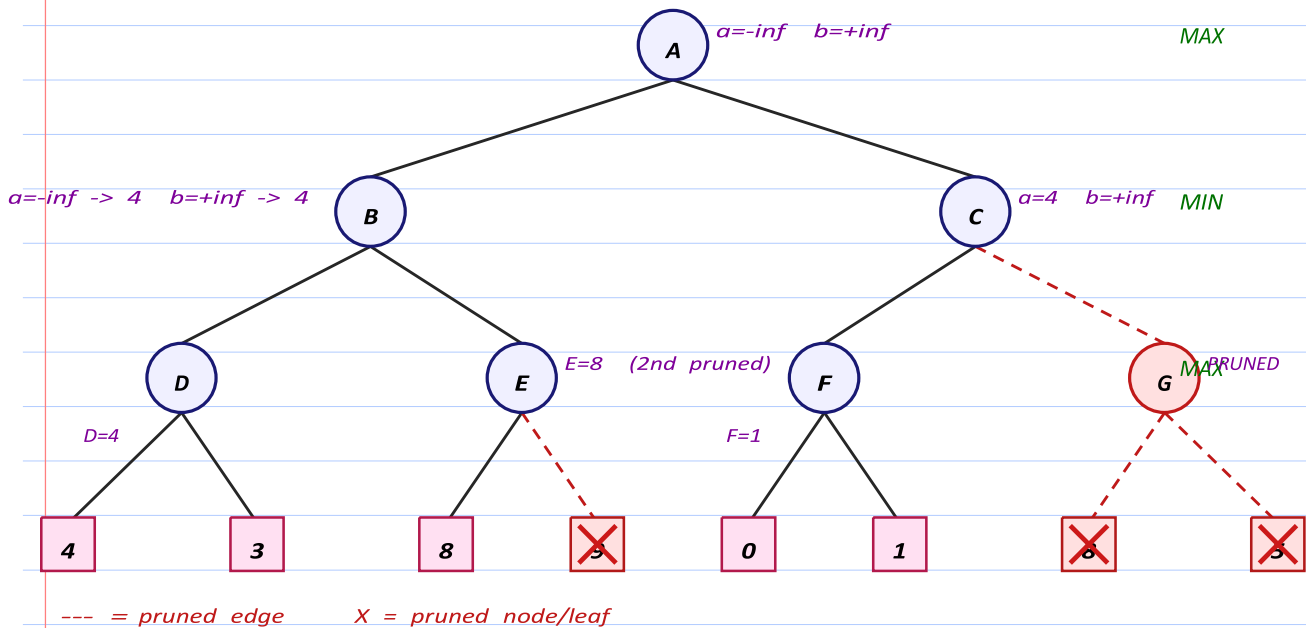
Q19. How do you call minimax for the first move?

Ans: `best_val = minimax(root_node, max_depth, True)` # True = Maximizer starts

Q20. What is the difference between minimax and alpha-beta?

Ans: Alpha-beta pruning skips branches that cannot affect the final decision, maki

Q1. Apply Alpha-Beta (α - β) Pruning on the following graph



Step-by-step Alpha-Beta Pruning Trace:

Initial call: A is MAX, $\alpha = -\text{inf}$, $\beta = +\text{inf}$

Step 1: Visit B (MIN). $\alpha = -\text{inf}$, $\beta = +\text{inf}$ Visit D (MAX): children

4, 3.

$D = \max(4, 3) = 4$.

B updates beta: $\beta = \min(+\text{inf}, 4) = 4$

Step 2: Visit E (MAX). 1st child = 8.

$E_{\text{best}} = 8$. Check: $E_{\text{best}}(8) \geq B$'s $\beta(4)$. PRUNE E's 2nd child

(value=9). [beta cutoff] $E = 8$. But $B = \min(4, 8) = 4$.

Step 3: $B = 4$. Return to A.

A updates alpha: $\alpha = \max(-\text{inf}, 4) = 4$.

Step 4: Visit C (MIN). $\alpha = 4$, $\beta = +\text{inf}$.

Visit F (MAX): children 0, 1.

$F = \max(0, 1) = 1$.

$C_{\text{best}} = \min(+\text{inf}, 1) = 1$.

Check: $C_{\text{best}}(1) \leq A$'s $\alpha(4)$. [alpha cutoff] PRUNE G entirely (children 8, 5 never visited).

Step 5: $C = 1$. $A = \max(4, 1) = 4$.

==> Optimal value at root A = 4

Pruned: leaf(9), node G, leaf(8b), leaf(5)

Task # 2 Q3. 20 Questions with Answers

(From Alpha-Beta Pruning Code and Concepts)

Q1. What is alpha-beta pruning?

Ans: It is an optimization of minimax that prunes branches which cannot affect the final decision, saving computation time.

Q2. What does the variable 'alpha' represent?

Ans: Alpha is the best (highest) value the Maximizer can guarantee along the current path from root.

Q3. What does the variable 'beta' represent?

Ans: Beta is the best (lowest) value the Minimizer can guarantee along the current path from root.

Q4. Write the pruning condition for alpha-beta.

Ans: if $\alpha \geq \beta$: break This means the current branch cannot improve the result.

Q5. Write the syntax/signature of the alpha-beta function.

```
Ans: def alphabeta(node, depth, alpha, beta, isMaximizing):  
    if depth==0 or is_terminal(node): return evaluate(node) if isMaximizing: val =  
    -infinity for child in get_children(node):  
        val = max(val, alphabeta(child,depth-1,alpha,beta,False)) alpha = max(alpha, val)  
        if alpha >= beta: break # beta cutoff return val else:  
        val = +infinity for child in get_children(node):  
            val = min(val, alphabeta(child,depth-1,alpha,beta,True)) beta = min(beta, val)  
            if alpha >= beta: break # alpha cutoff return val
```

Q6. What are the initial values of alpha and beta?

Ans: $\alpha = -\text{infinity}$ (worst for Maximizer) $\beta = +\text{infinity}$ (worst for Minimizer)

Q7. Does alpha-beta produce the same result as minimax?

Ans: Yes. Alpha-beta always produces the exact same optimal value as plain minimax, just faster by skipping useless branches.

Q8. What is the best-case time complexity of alpha-beta?

Ans: $O(b^{(d/2)})$ when moves are ordered best-first.
This effectively doubles the search depth versus minimax.

Q9. What is the worst-case time complexity of alpha-beta?

Ans: $O(b^d)$ — same as minimax when moves are ordered worst-first and no pruning occurs.

Q10. What is a beta cutoff?

Ans: At a MAX node, when $\alpha \geq \beta$, the Minimizer above will never choose this path.
Remaining children are pruned.

Q11. What is an alpha cutoff?

Ans: At a MIN node, when $\beta \leq \alpha$, the Maximizer above will never choose this path.
Remaining children are pruned.

Q12. Which nodes were pruned in the given tree (Task 2)?

Ans: Leaf(9) under E was pruned (beta cutoff at B).
Node G and its leaves (8,5) were pruned (alpha cutoff at C).

Q13. What is the final result at root A in the given tree?

Ans: Root A = 4. The Maximizer gets value 4 via path A → B → D.

Q14. Why was leaf 9 (under E) pruned?

Ans: Because B's beta was 4 (from D). E's first child gave $8 \geq 4$, so B cannot do better than 4.
Second child (9) is irrelevant.

Q15. Why was node G pruned entirely?

Ans: C already had value 1 from F. Since $1 \leq \alpha(4)$ of A, A (Maximizer) will never pick C. So
G is pointless to visit.

Q16. How does move ordering improve alpha-beta efficiency?

Ans: Evaluating best moves first creates tighter alpha/beta bounds earlier, leading to more
pruning and closer to $O(b^{d/2})$.

Q17. What is the call syntax to start alpha-beta search?

Ans: `import math`
`result = alphabeta(root, max_depth, -math.inf, math.inf, True)`

Q18. What does 'isMaximizing=True' mean in the call?

Ans: It means the root node is the Maximizer's turn, so we are
looking for the maximum possible outcome.

Q19. Can alpha and beta be passed by reference or value?

Ans: In Python they are passed by value. Each recursive call gets its own copy, so parent's
alpha/beta must be updated explicitly.

alpha-beta better than plain minimax?

Q20.
Why is

Ans: It reaches the same optimal decision while exploring far fewer nodes, saving both time and memory in game-tree search.